Convolutional Neural Networks

Alexandru Sorici

October 16, 2024

Inspiration for some slides: Fei-Fei Li, Ehsan Adeli, Stanford CS231n Lectures on CNNs

Table of contents

1. Introduction

2. Building blocks of CNNs

3. Normalization

4. Classic Networks

5. Regularization in Conv Nets

6. Transfer Learning

Intro

Conv Nets are everywhere (contd)

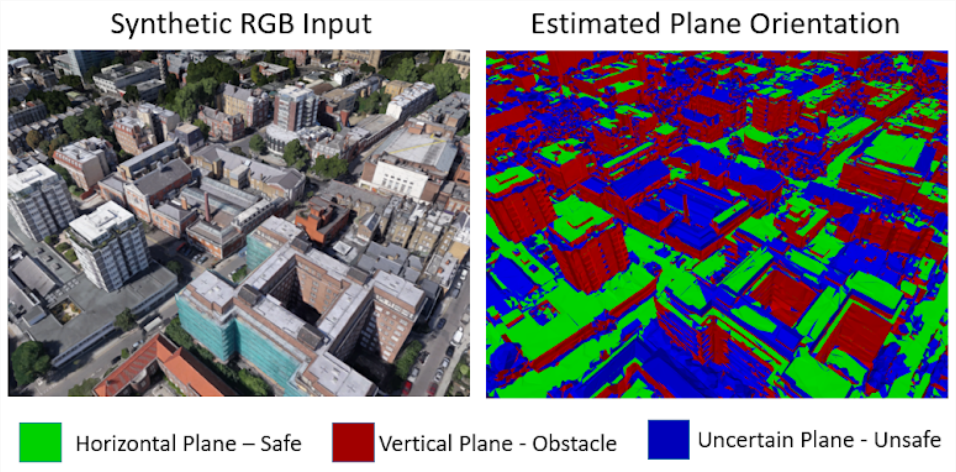


Figure 1: SafeUAV: Learning to estimate depth and safe landing areas for UAVs from synthetic data. Alina Marcu, Dragos Costea, Vlad Licaret, Mihai Parvu, Marius Leordeanu

Conv Nets are everywhere (contd)

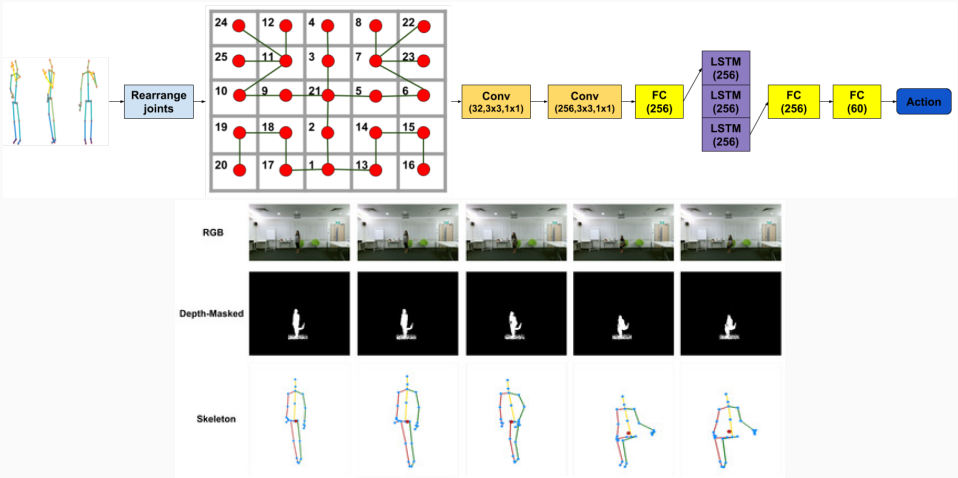


Figure 2: Spatio-Temporal Features in Action Recognition Using 3D Skeletal Joints. M. Trascau, M. Nan, AM Florea

Conv Nets are everywhere(contd)

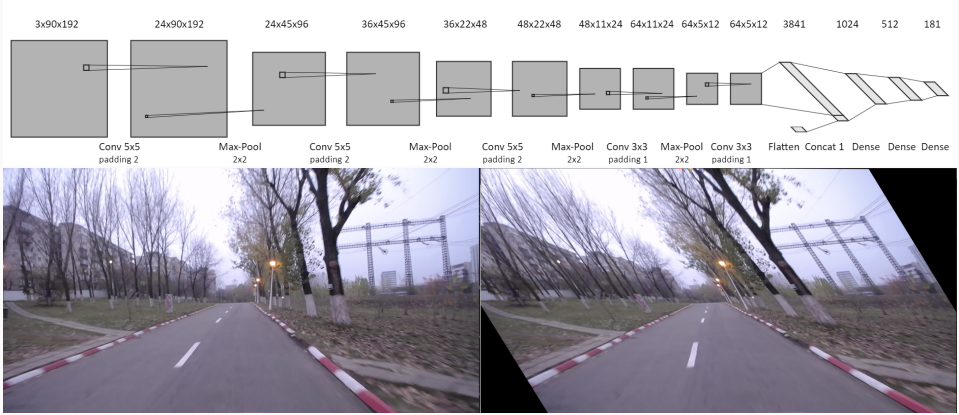


Figure 3: End-to-end models for self-driving cars on UPB campus roads. A. Mihalea, R. Samoilescu, A. Nica, M. Trascau, A. Sorici, AM Florea

CNN Building Blocks

Convolution Operation

For the continuous case, a convolution product between two functions f and g is defined as:

$$(f * g)(t) = \int_{-\infty}^{+\infty} f(\tau)g(t-\tau)d\tau = \int_{-\infty}^{\infty} f(t-\tau)g(\tau)d\tau$$

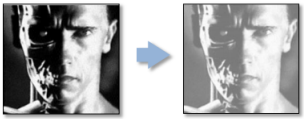
For the discrete case, integral is replaced by a sum:

$$(f * g)(n) = \sum_{m=-\infty}^{\infty} f(m)g(n-m) = \sum_{m=-\infty}^{\infty} f(n-m)g(m)$$

Convolution Operation in spatial domain

An **RGB** image can be considered as a function ($f : \mathbb{R}^2 \rightarrow \mathbb{R}^3$), where $f(x, y) = (red_value, green_value, blue_value)^T$

Operators can be applied to the image, by treating is a function:



$g(x,y) = f(x,y) + 20$



$g(x,y) = f(-x,y)$

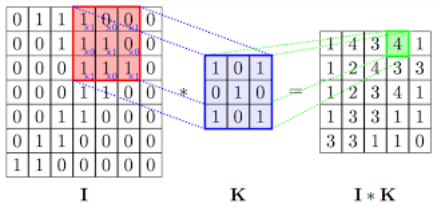
Convolution Operation in spatial domain

Convolution Operation on images

The convolution operation implies use of a convolution kernel (K - of dimension $w \times w$, where $w = 2k + 1$), applied to source image (I):

$$O = I * K$$

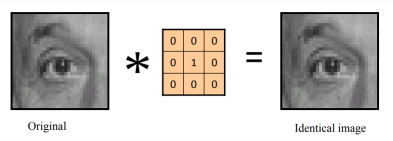
$$O(i,j) = \sum_{u=0}^{w-1} \sum_{v=0}^{w-1} I(i+u-k, j+v-k) \cdot K(u,v)$$

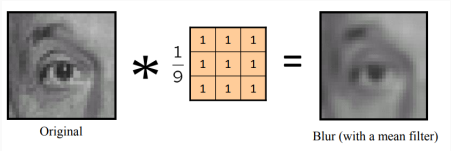


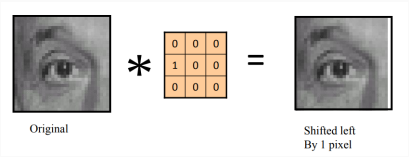
Convolution Operation in spatial domain - contd

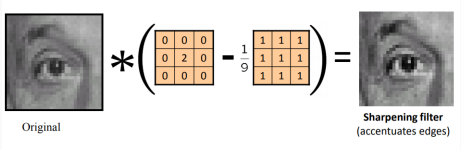
Properties

Conv. op. is **associative** and **comutative**.





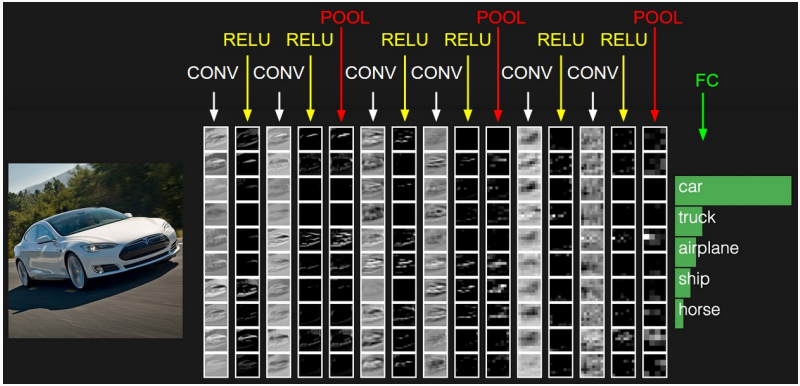




CNN Layers

We use 3 main types of layers when constructing CNN architectures:

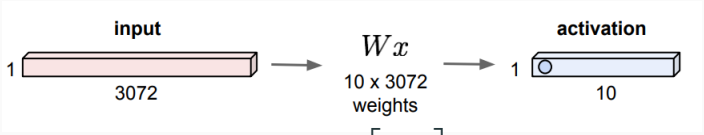
- **Convolution Layer**
- **Pooling Layer**
- **Fully Connected Layer** (like in MLP)



Fully Connected Layer

Consider an image of dim. $32 \times 32 \times 3$. We wish to ascribe it to *one* of 10 classes.

1. Linearize image $\implies$ dimension of 3072×1.
2. Pass resulting vector through a fully connected layer.



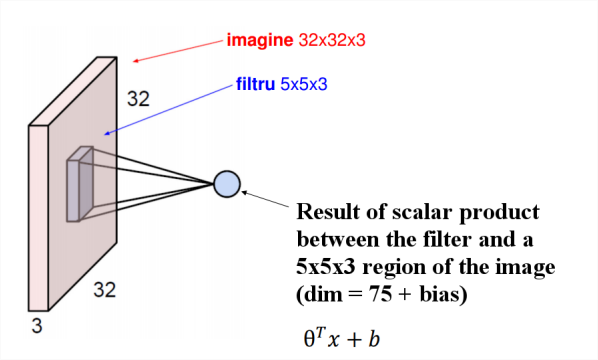
$$\begin{bmatrix} w_{1,1} & w_{1,2} & \dots & w_{1,3072} \\ \vdots & \vdots & \vdots & \vdots \\ w_{10,1} & w_{10,2} & \dots & w_{10,3072} \end{bmatrix} \cdot \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_{3072} \end{bmatrix} + \begin{bmatrix} b_1 \\ \vdots \\ b_{10} \end{bmatrix} = \begin{bmatrix} y_1 \\ \vdots \\ y_{10} \end{bmatrix} \tag{1}$$

$$H(X) = (W.x) + b^T$$

Disadvantage: lose info about spatial arrangement of pixel (example of inductive bias)

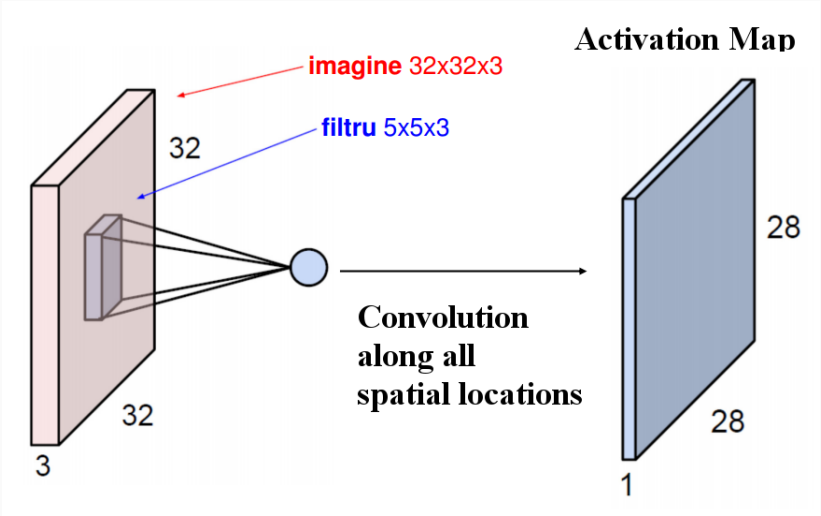
Convolution Layer - I

Alternative: connect each neuron to only a *local* region of the input volume

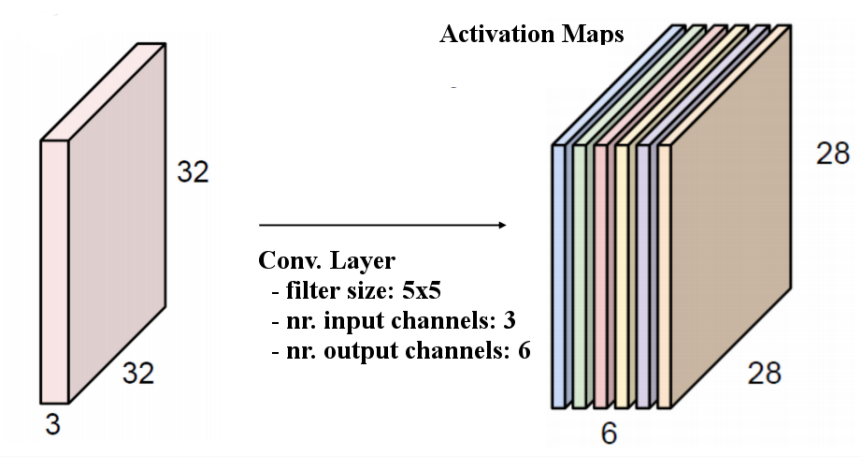


Receptive Field = *size of the filter* - spatial extent of the *local* connectivity of each neuron

Convolution Layer - I



Convolution Layer - I



Convolution Layer - II

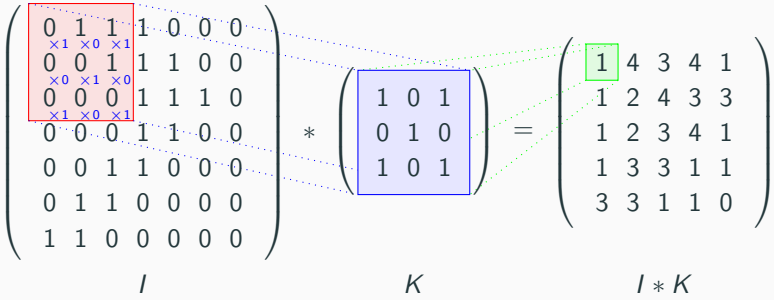
Case study

For an input volume (e.g. source image) of $32 \times 32 \times 3$ and apply a filter of size $5 \times 5 \times 3$ the dimension of the output map is: ?

Convolution Layer - II

Case study

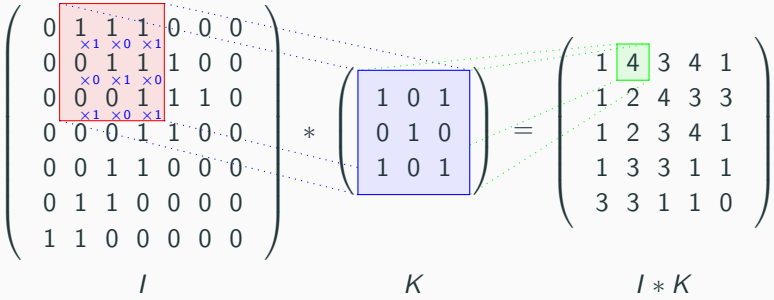
For an input volume (e.g. source image) of $32 \times 32 \times 3$ and apply a filter of size $5 \times 5 \times 3$ the dimension of the output map is: $28 \times 28 \times 1$



Convolution Layer - II

Case study

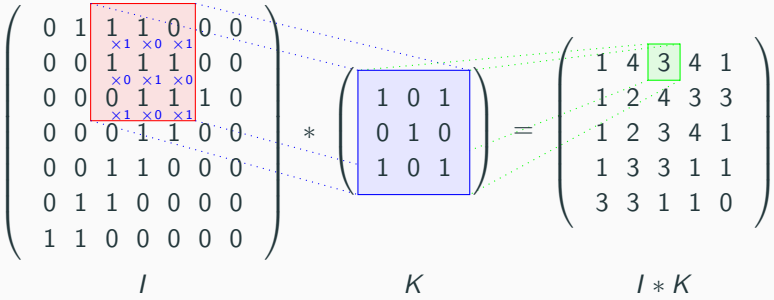
For an input volume (e.g. source image) of $32 \times 32 \times 3$ and apply a filter of size $5 \times 5 \times 3$ the dimension of the output map is: $28 \times 28 \times 1$



Convolution Layer - II

Case study

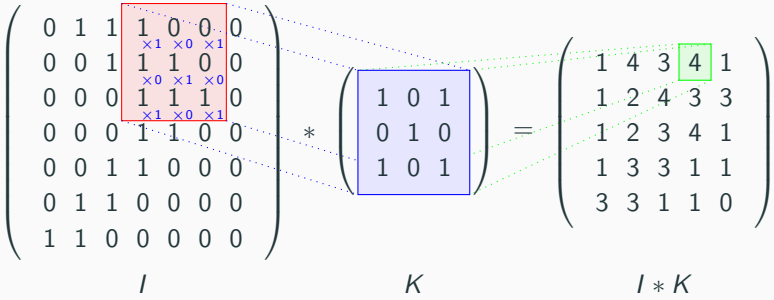
For an input volume (e.g. source image) of $32 \times 32 \times 3$ and apply a filter of size $5 \times 5 \times 3$ the dimension of the output map is: $28 \times 28 \times 1$



Convolution Layer - II

Case study

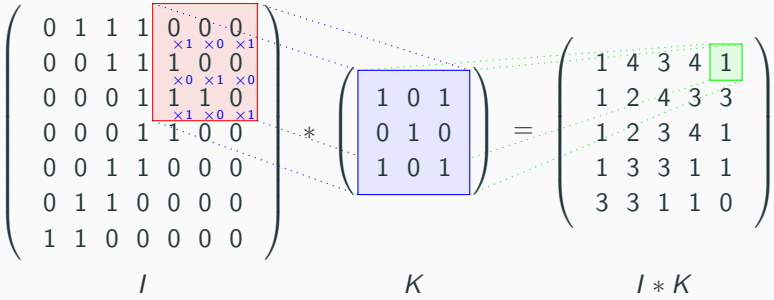
For an input volume (e.g. source image) of $32 \times 32 \times 3$ and apply a filter of size $5 \times 5 \times 3$ the dimension of the output map is: $28 \times 28 \times 1$



Convolution Layer - II

Case study

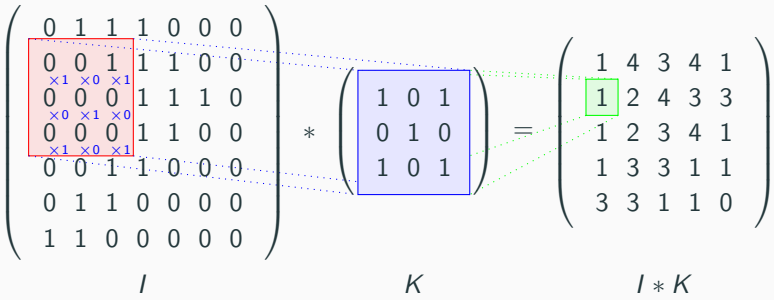
For an input volume (e.g. source image) of $32 \times 32 \times 3$ and apply a filter of size $5 \times 5 \times 3$ the dimension of the output map is: $28 \times 28 \times 1$



Convolution Layer - II

Case study

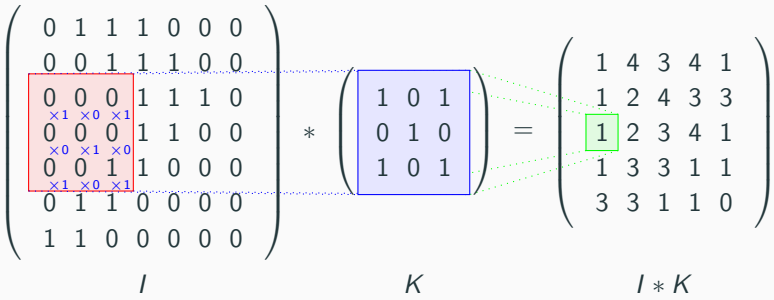
For an input volume (e.g. source image) of $32 \times 32 \times 3$ and apply a filter of size $5 \times 5 \times 3$ the dimension of the output map is: $28 \times 28 \times 1$



Convolution Layer - II

Case study

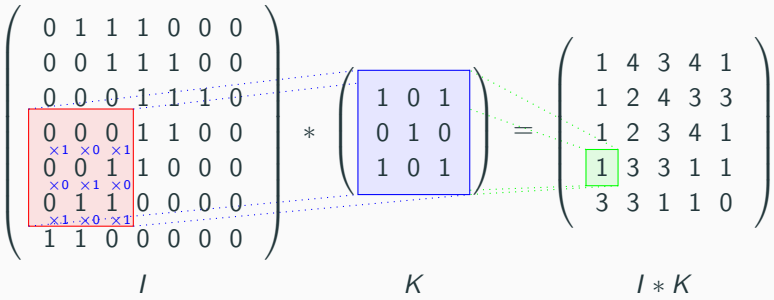
For an input volume (e.g. source image) of $32 \times 32 \times 3$ and apply a filter of size $5 \times 5 \times 3$ the dimension of the output map is: $28 \times 28 \times 1$



Convolution Layer - II

Case study

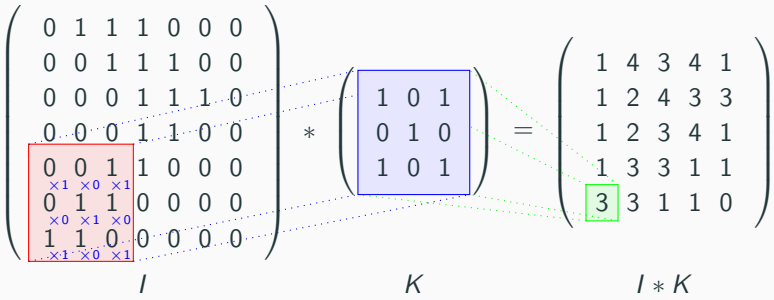
For an input volume (e.g. source image) of $32 \times 32 \times 3$ and apply a filter of size $5 \times 5 \times 3$ the dimension of the output map is: $28 \times 28 \times 1$



Convolution Layer - II

Case study

For an input volume (e.g. source image) of $32 \times 32 \times 3$ and apply a filter of size $5 \times 5 \times 3$ the dimension of the output map is: $28 \times 28 \times 1$



Convolution Layer - II

Case study

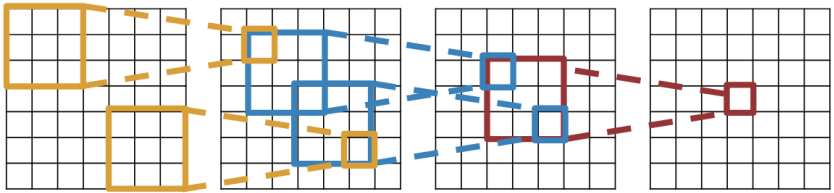
For an input volume (e.g. source image) of $32 \times 32 \times 3$ and apply a filter of size $5 \times 5 \times 3$ the dimension of the output map is: $28 \times 28 \times 1$

General case

Starting from an input volume of dimension: $Height \times Width \times Depth$ and applying a convolution layer with kernel (filter) of size $K \times K \times Depth$ we get an activation map of size $[Height - (K - 1)] \times [Width - (K - 1)] \times 1$.

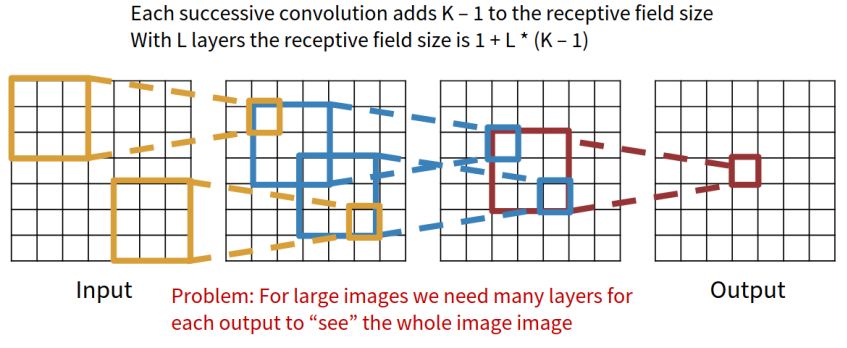
Convolution Layer - Receptive fields

Each successive convolution adds K – 1 to the receptive field size
With L layers the receptive field size is 1 + L * (K – 1)



Problem: For large images we need many layers for each output to “see” the whole image image

Convolution Layer - Receptive fields



- Solution to increasing receptive field: downsample *within* the network
  - Control application of the kernel
  - Reduce height x width resolution of the input volume

Convolution Layer - III - HyperParameters

Four hyperparameters control the size of the output volume:

- **Depth**: number of filters (each looks at smth different in the input).
- **Stride**: the step taken when *sliding* the filter. Usual practice - stride $= 1$ or 2 (3 or more very uncommon)
- **Zero-Padding**: size of the number of 0s that surround the border of the input volume. Most common use: *preserve spatial size of input volume*, i.e. input and output have the same *width* and *height*.
- **Dilation**: distance in between elements of the convolution kernel.

Convolution Layer - III - HyperParameters

Convolution arithmetic animation :-)

Source: https://github.com/vdumoulin/conv_arithmetic

Convolution Layer - III - HyperParameter arithmetic

Convolution output size computation
How do *kernel size*, *stride*, *padding*, *dilation* influence the size of the output?

Convolution Layer - III - HyperParameter arithmetic

Convolution output size computation

How do *kernel size*, *stride*, *padding*, *dilation* influence the size of the output?

Formulae

$$H_{out} = \left[\frac{H_{in} + 2 * padding - dilation * (kernel_size - 1) - 1}{stride} + 1 \right]$$

$$W_{out} = \left[\frac{W_{in} + 2 * padding - dilation * (kernel_size - 1) - 1}{stride} + 1 \right]$$

Observation

There is a *constraint* on strides: result of the division has to be an integer. E.g. For input of size $H_{in} = 10$, $padding = 0$, $kernel_size = 3$, $dilation = 1$, use of $stride = 2$ is not possible because:
$\left[\frac{H_{in}+2*padding-dilation*(kernel_size-1)-1}{stride} + 1 \right] = (10 + 2 * 0 - 1 * (3 - 1) - 1)/2 + 1 = 7/2 + 1,$
which is not an integer

Convolution Layer - III - Parameter Sharing

Parameter sharing is used in CNNs to control the number of learnable parameters.
E.g. if output volume has $55 \times 55 \times 96$ and filter size is $11 \times 11 \times 3$, having a weight for each neuron would result in $290400 \times 364 = 105705600$ parameters **just for one layer**.

$\implies$ Make a simplifying assumption: *if one feature is useful to compute at some spatial position (x,y), then it should also be useful to compute at a different position (x2,y2).* $\implies$

Parameters *within a depth slice* are shared: $11 \times 11 \times 3 \times 96 = 34944$ ($+$ 96 biases).

Convolution Layer - III - Learned Features

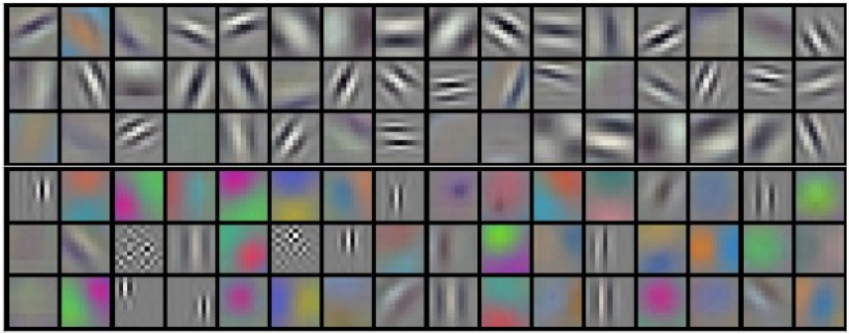


Figure 5: Example filters learned by Krizhevsky et al. Each of the 96 filters shown here is of size [11x11x3], and each one is shared by the 55*55 neurons in one depth slice. Notice that the parameter sharing assumption is relatively reasonable: If detecting a horizontal edge is important at some location in the image, it should intuitively be useful at some other location as well due to the translationally-invariant structure of images. There is therefore no need to relearn to detect a horizontal edge at every one of the 55*55 distinct locations in the Conv layer output volume. Source: Stanford CS321n notes.

Convolution Layer - IV - PyTorch classes

- **Class Header**: `class torch.nn.Conv1d(in_channels, out_channels, kernel_size, stride=1, padding=0, dilation=1, groups=1, bias=True)`
- **Input**: (N, C_{in}, L_{in})
- **Output**: (N, C_{out}, L_{out}) $L_{out} = \left[\frac{L_{in} + 2 * padding - dilation * (kernel_size - 1) - 1}{stride} \right] + 1$
- **Formula**:

$$out(N_i, C_{out_j}) = bias(C_{out_j}) + \sum_{k=0}^{C_{in}-1} weight(C_{out_j}, k) \star input(N_i, k)$$

Convolution Layer - IV - Pytorch Conv2D

- **Class Header**: `class torch.nn.Conv2d(in_channels, out_channels, kernel_size, stride=1, padding=0, dilation=1, groups=1, bias=True)`
- **Input**: $(N, C_{in}, H_{in}, W_{in})$
- **Output**: $(N, C_{out}, H_{out}, W_{out})$

  $H_{out} = \left[\frac{H_{in}+2*padding[0]-dilation[0]*(kernel_size[0]-1)-1}{stride[0]}\right] + 1$

  $W_{out} = \left[\frac{W_{in}+2*padding[1]-dilation[1]*(kernel_size[1]-1)-1}{stride[1]}\right] + 1$
- **Formula**:

$$out(N_i, C_{out_j}) = bias(C_{out_j}) + \sum_{k=0}^{C_{in}-1} weight(C_{out_j}, k) \star input(N_i, k)$$

Convolution Layer - IV - Pytorch Conv3D

- **Class Header**: `class torch.nn.Conv3d(in_channels, out_channels, kernel_size, stride=1, padding=0, dilation=1, groups=1, bias=True)`
- **Input**: $(N, C_{in}, D_{in}, H_{in}, W_{in})$
- **Output**: $(N, C_{out}, D_{out}, H_{out}, W_{out})$

$$D_{out} = \left\lfloor \frac{D_{in} + 2 * padding[0] - dilation[0] * (kernel_size[0] - 1) - 1}{stride[0]} \right\rfloor + 1$$

$$H_{out} = \left\lfloor \frac{H_{in} + 2 * padding[1] - dilation[1] * (kernel_size[1] - 1) - 1}{stride[1]} \right\rfloor + 1$$

$$W_{out} = \left\lfloor \frac{W_{in} + 2 * padding[2] - dilation[2] * (kernel_size[2] - 1) - 1}{stride[2]} \right\rfloor + 1$$

- **Formula**:

$$out(N_i, C_{out_j}) = bias(C_{out_j}) + \sum_{k=0}^{C_{in}-1} weight(C_{out_j}, k) \star input(N_i, k)$$

Convolution Layer - IV - Grouped Convolutions

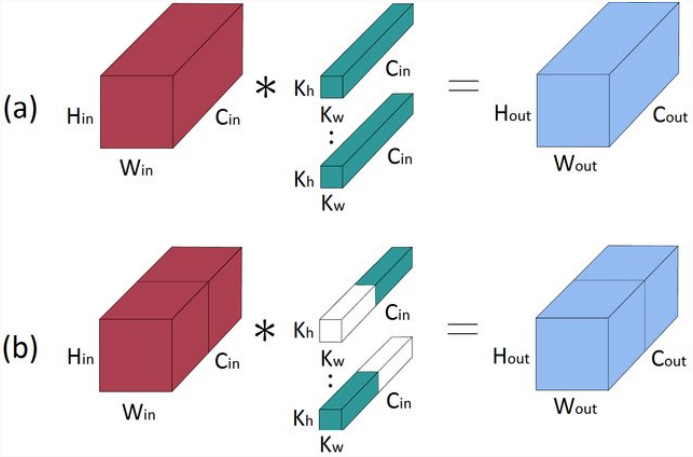


Figure 6: a) Normal convolution, b) Grouped Convolution with 2 groups. Source: Optimizing Grouped Convolutions on Edge Devices, Gibson et al. 2018

Convolution Layer - V - Depthwise Separable Convolutions

- Separate the convolution operation into two separate operations: (i) a *depthwise* convolution and (ii) a *pointwise* convolution

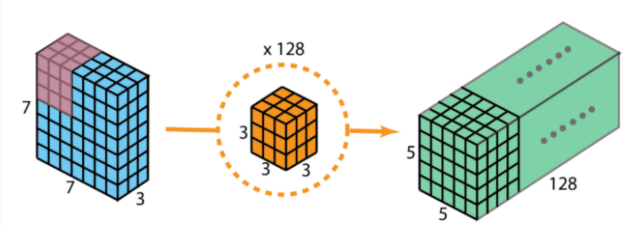


Figure 7: Typical 2D convolution. img. src: opengenus.org

- No. multiplications for typical 2D conv: 128 3x3x3 kernels moving 5x5 times $\implies$ 128 x 3 x x 3 x 5 x 5 = 86400

Convolution Layer - V - Depthwise Separable Convolutions (cont)

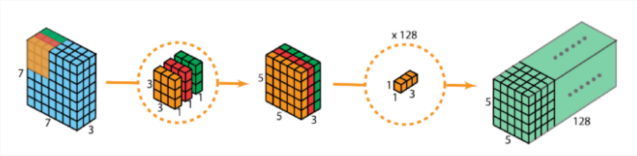


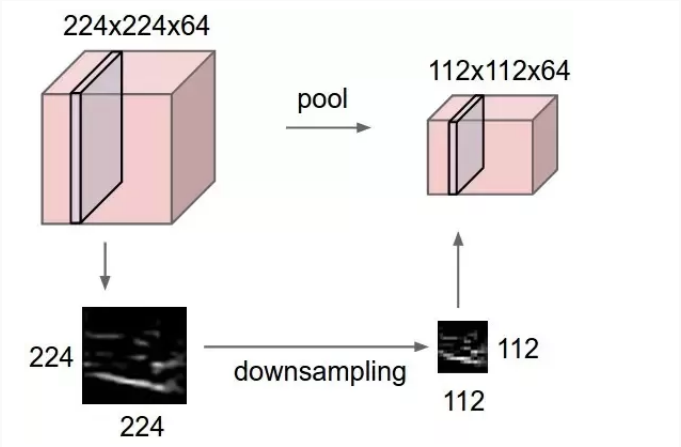
Figure 8: Composition of depth- and point-wise convolutions. img. src: opengenus.org

Computational advantage

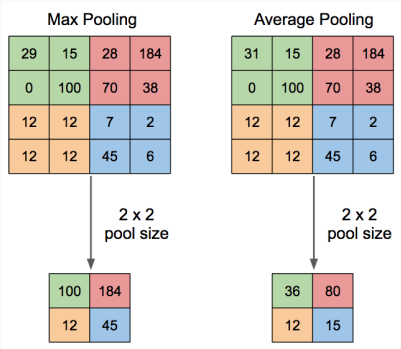
- depthwise: 3 3x3x1 kernels moving 5 x 5 times $\implies$ 3 x 3 x 3 x 1 x 5 x 5 = 675
- pointwise: 128 1x1x3 filters moving 5 x 5 times $\implies$ 128 x 1 x 1 x 3 x 5 x 5 = 9600
- In general, reduction is proportional to $\frac{1}{k^2}$, where k is size of kernel
- Depthwise separable convolution used in small, efficient conv nets (e.g. MobileNet suite)

Pooling Layer

- Modifies the input volume into a *smaller* and *more manageable* representation
- Operates independently over each activation map



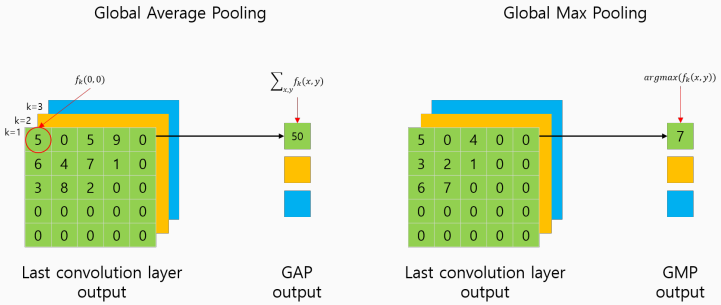
Pooling Layer



Common settings:

- filter size $= 2$, stride $= 2$
- filter size $= 3$, stride $= 2$

Pooling Layer



- Used usually at final stages of a conv net architecture to replace fully connected layers
- Tries to avoid overfitting by forcing feature maps to hold "global" information that is relevant for the subsequent task (e.g. classification, identification)

Normalization

Normalization

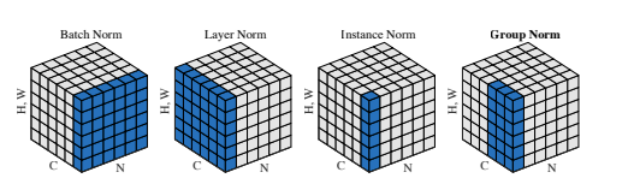
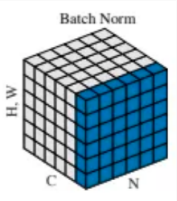


Figure 9: Visual comparison of different normalization methods. N - batch dimension, C - channel (feature map) dimension, H,W - input size dimension

Uses of normalization

- address *covariate-shift* problem - simplification of the learning dynamic $\implies$ higher learning rates are possible $\implies$ faster convergence time
- makes the loss surface smoother (magnitude of gradients bounded better $\implies$ "better behaved" gradients)
- some form of normalization becomes necessary in very deep networks

Batch Normalization



γ and β are *learnable* parameters $\rightarrow$ mean and variance of layer activations are *decided* by two simple parameters (as opposed to complex interactions between layers during learning)

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1...m}\}$;
Parameters to be learned: γ, β
Output: $\{y_i = \mathrm{BN}_{\gamma,\beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m} x_i \qquad \text{// mini-batch mean}$$

$$\sigma^2_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_{\mathcal{B}})^2 \qquad \text{// mini-batch variance}$$

$$\widehat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma^2_{\mathcal{B}} + \epsilon}} \qquad \text{// normalize}$$

$$y_i \leftarrow \gamma \widehat{x}_i + \beta \equiv \mathrm{BN}_{\gamma,\beta}(x_i) \qquad \text{// scale and shift}$$

Batch Normalization - Discussion

Batch Normalization caveats

- Normalization statistics in BatchNorm correspond to *mean* and *variance* of the **mini-batch** (instead of the whole data set)
- small mini-batches may produce noisy estimates $\Rightarrow$ negatively affects training

Batch Normalization - Discussion

Batch Normalization caveats

- Normalization statistics in BatchNorm correspond to *mean* and *variance* of the **mini-batch** (instead of the whole data set)
- small mini-batches may produce noisy estimates $\Rightarrow$ negatively affects training

Batch Normalization during Fine Tuning

- Question: use the statistics of mean and variance from the *original dataset* or the *new* one?
- Most frameworks *do not* freeze the BatchNorm params (γ, β) when freezing a layer $\Rightarrow$ they use the mini-batch statistics from the *new* dataset
- If new dataset is large and mini-batch size is comparable (equal) to original dataset $\rightarrow$ can retrain statistics
- If new dataset is small or mini-batch size is small $\rightarrow$ may be better to use original dataset statistics

Classic Networks

Classic Networks - LeNet 5

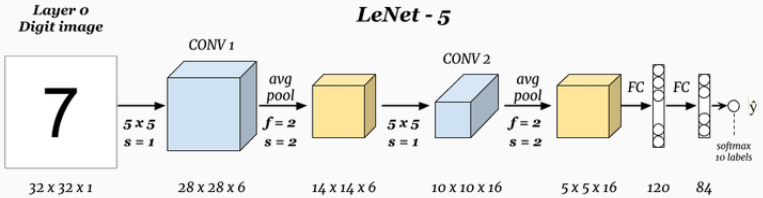


Figure 10: LeNet-5 - Y. Lecun, L. Bottou, Y. Bengio, P. Haffner (**1998**)

- **No. params:** 60000

Classic Networks - AlexNet

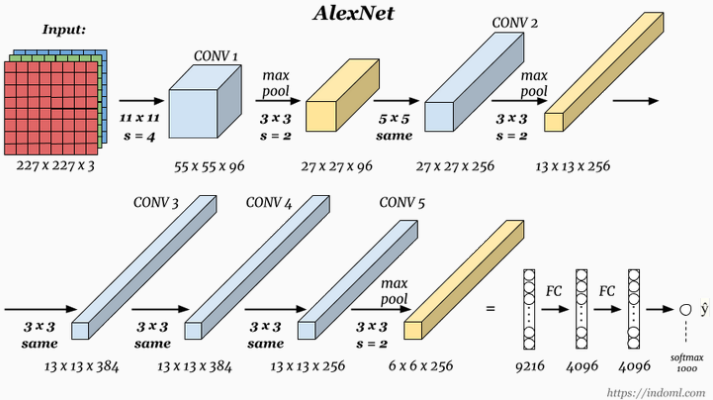


Figure 11: AlexNet - Alex Krizhevsky, Geoffrey Hinton, Ilya Sutskever (2012)

- **No. params:** 60 million

Classic Networks - AlexNet

Details / Retrospectives:

- First CNN to win ImageNet challenge
- First use of ReLU
- heavy data augmentation (lots of params to train)
- dropout of 0.5
- batch size of 128
- optimization using SGD + Momentum 0.9
- learning rate 1e-2, reduced by 10 manually when accuracy plateaus
- L2 weight decay 5e-4
- in competition used 7 CNN ensemble: 18.2% $\rightarrow$ 15.2%

Classic Networks - VGG-16

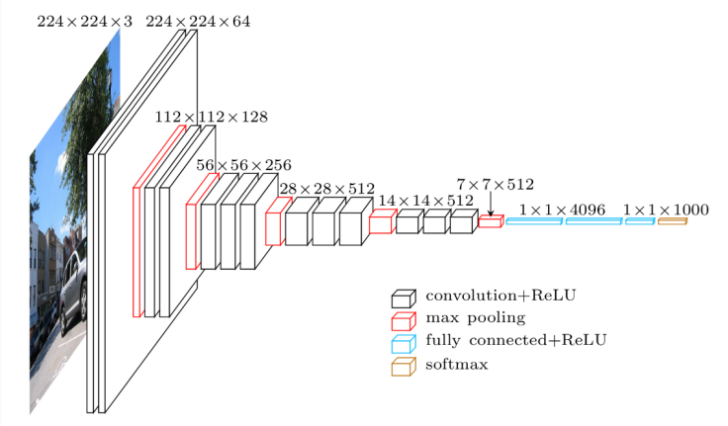
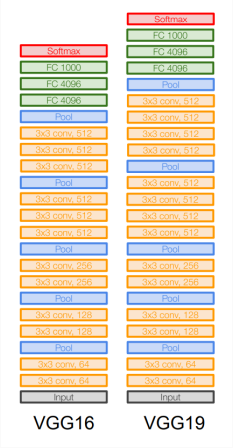


Figure 12: VGG-16 - Karen Simonyan and Andrew Zisserman (2014)

- **No. params:** 138 million

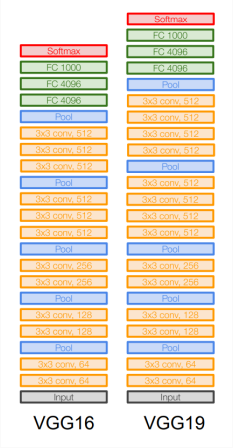
Classic Networks - VGG-16



- Uses only 3×3 CONV, stride 1, pad 1 and 2×2 MAX POOL stride 2

Figure 13: Block diagram of VGG-16 and VGG-19.
Source: Stanford CS231n Lecture 9 - slide 42

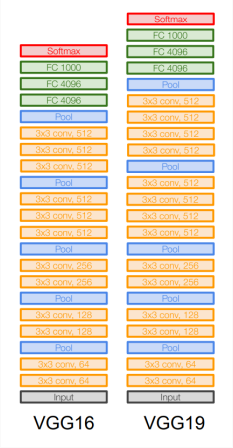
Classic Networks - VGG-16



- Uses only 3×3 CONV, stride 1, pad 1 and 2×2 MAX POOL stride 2
- Stack of 3×3 conv (stride 1) layers has same effectve receptive field as one 7×7 conv layer ... but *deeper*, *more non-linearities* and *fewer params:* $3 \times (3^2 C^2)$ vs $7^2 C^2$, where C is channels per layer

Figure 13: Block diagram of VGG-16 and VGG-19.
Source: Stanford CS231n Lecture 9 - slide 42

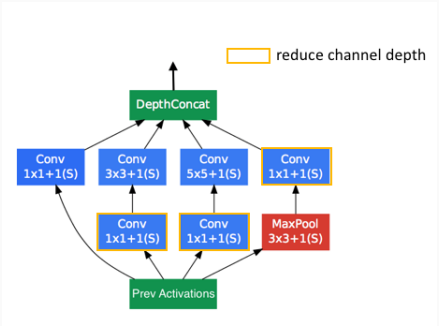
Classic Networks - VGG-16

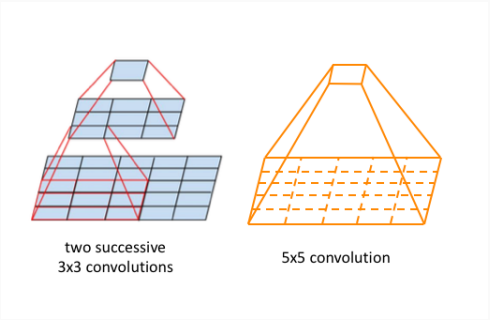


- Uses only 3×3 CONV, stride 1, pad 1 and 2×2 MAX POOL stride 2
- Stack of 3×3 conv (stride 1) layers has same effectve receptive field as one 7×7 conv layer ... but *deeper*, *more non-linearities* and *fewer params:* $3 \times (3^2 C^2)$ vs $7^2 C^2$, where C is channels per layer
- Similar training procedure as AlexNet
- FC7 features generalize well to other tasks

Figure 13: Block diagram of VGG-16 and VGG-19.
Source: Stanford CS231n Lecture 9 - slide 42

Classic Networks - GoogleNet - Inception Cell





Classic Networks - GoogleNet - Architecture

Details:

- Introduces *bottleneck* layers, using 1×1 convolutions to reduce computational complexity (reduce number of channels over which 3×3 and 5×5 conv. layers have to operate)
- Philosophy of "inception module": design good local network topology and *stack* the modules on top of another
- Apply parallel filter operations on the same input
  - Multiple receptive field sizes (1×1, 3×3, 5×5,
  - Pooling operation (3×3)
- Concatenate filter outputs depth-wise

Classic Networks - GoogleNet - Architecture

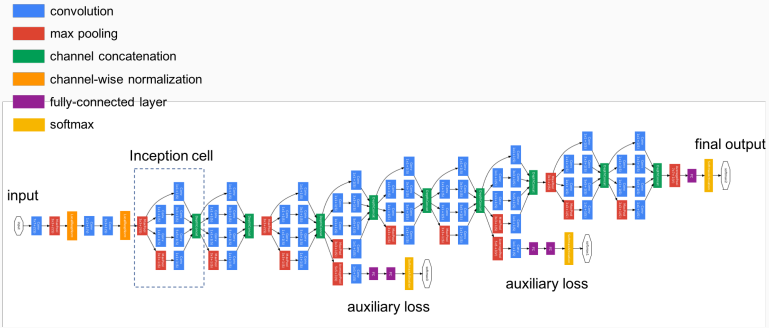


Figure 14: GoogleNet - Christian Szegedy et al (2014)

- **No. params:** 5 million

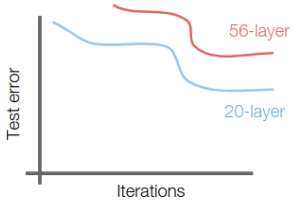
Classic Networks - GoogleNet - Architecture

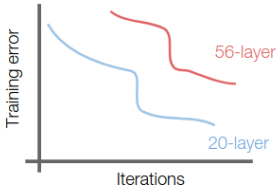
Details

- After the last convolutional layer, a *global* average pooling layer is used that spatially averages across each feature map, before final FC layer.
- No longer multiple expensive FC layers
- Uses auxilliary classification outputs to inject additional gradient at lower layers

Classic Networks - Residual Network (ResNet) - Skip connection

What happens when we continue stacking deeper layers on a “plain” convolutional neural network?

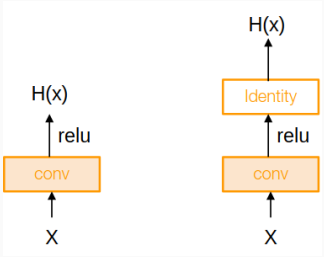




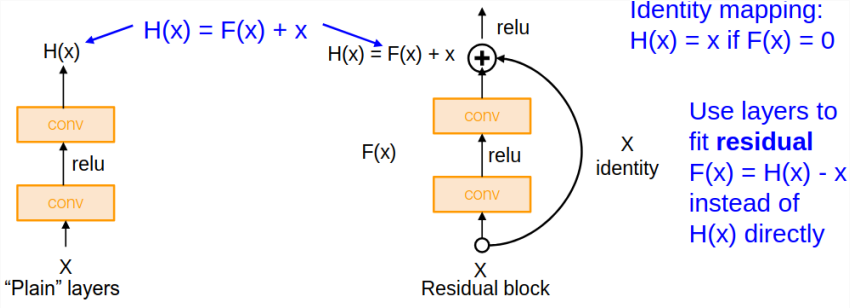
56-layer model performs worse on both test and training error
-> The deeper model performs worse, but it’s not caused by overfitting!

Classic Networks - Residual Network (ResNet) - Skip connection

- We know that more layers = more parameters = higher representation power
- The problem in deeper networks is an **optimization** one $\Rightarrow$ deeper models are harder to optimize
- If shallow models *learn* a valuable representation, a solution *by construction* of a deeper model would be to *copy* the learned layers from a shallow model and set the rest of the layers to identity mapping



Classic Networks - Residual Network (ResNet) - Skip connection



- Solution in ResNet: use convolution layers to *fit* a **residual** mapping ($F(x)$) instead of a directly trying to fit an underlying mapping ($H(x)$)

Classic Networks - Residual Network (ResNet) - Skip connection

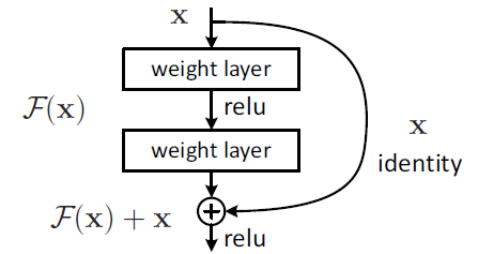


Figure 15: Skip Connection

$$y = x + \mathcal{F}(x) \Rightarrow \frac{\partial E}{\partial x} = \frac{\partial E}{\partial y} \cdot \frac{\partial y}{\partial x} = \frac{\partial E}{\partial y} \cdot (1 + \mathcal{F}'(x))$$

- Aid optimization by ensuring a *"highway"* for gradients to propagate backwards to closer-to-input layers ($\frac{\partial E}{\partial y} \cdot 1$)

Classic Networks - Residual Network (ResNet) - Skip connection

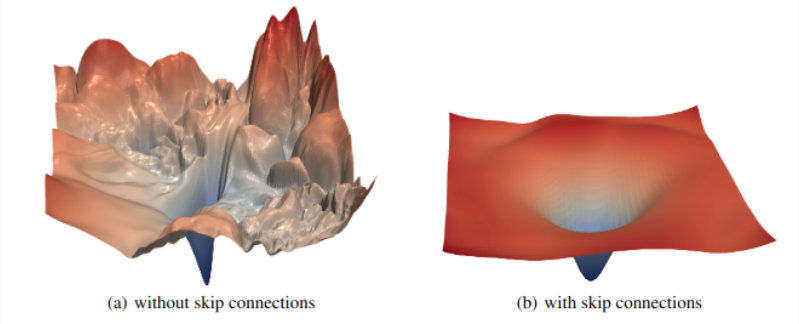
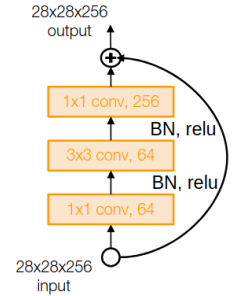


Figure 16: Image source: Li et al., Visualizing the Loss Landscape of Neural Nets, NeurIPS 2018

Classic Networks - Residual Network (ResNet) - Architecture

For deeper networks (ResNet-50+), use “bottleneck” layer to improve efficiency (similar to GoogLeNet)



- **Bottleneck layer** is used to reduce the *width* (number of channels in the input volume) that is used in the main 3x3 convolution inside a ResNet Block
- Example: input of 28 x 28 x 256 is "shrunk" to 28x28x64 (a 4x reduction) using a 64 channel 1x1 convolutio

Classic Networks

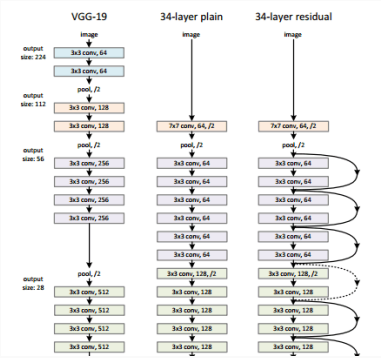


Figure 17: ResNet - Kaiming He et al (2015)

- **No. params:** 25 million (ResNet-50)
- ImageNet winner 2015: 3.6% error rate (*better than human performance - 5.1%)*

Classic Networks - Residual Network (ResNet) - Architecture

Details

- Issue: deep networks using stacks of "plain" conv layers suffer from optimization problem
- Solution: Use layers to fit *residual* $F(x) = H(x) - x$, instead of $H(x)$ directly (where $H(x)$ would be the output of the "plain" layers)
- Stack residual blocks; periodically double no. filters and downsample spatially using stride 2
- No FC layers at the end
- For deeper networks (ResNet-50+) use "bottleneck layer" to improve computational efficiency
- Training ResNet in practice:
  - BatchNorm after every CONV Layer, Xavier 2/ initialization
  - SGD + Momentum (0.9), Learning rate: 0.1 (divide by 10 when validation error plateaus)
  - Mini-batch 256, weight decay of 1e-5, no dropout

Classic Networks - Wide ResNet

Highlights

- Performs experiments to explore the problem of **diminishing feature reuse** (i.e. assumed weakness in original ResNet design, that can cause the network to forego using the residual blocks, because of skip connections)
- Experiments with
  - Adding Dropout regularization within the Residual Block
  - Different number of conv layers within a residual block (**l-factor**)
  - Multiplication of feature map size within residual block (**k-factor**)
  - conv-bn-relu vs bn-relu-conv (post-activation vs pre-activation)

Classic Networks - Wide ResNet

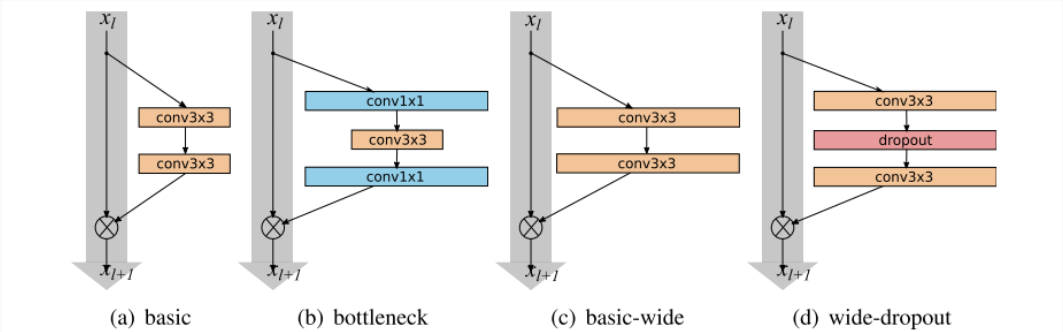


Figure 1: Various residual blocks used in the paper. Batch normalization and ReLU precede each convolution (omitted for clarity)

Figure 18: Source - Wide Residual Networks, Zagoruyko and Komodakis, 2017

Classic Networks - Wide ResNet

group name	output size	block type = $B(3,3)$
conv1	32×32	$[3\times3,\ 16]$
conv2	32×32	$\begin{bmatrix} 3\times3,\ 16\times k \\ 3\times3,\ 16\times k \end{bmatrix} \times N$
conv3	16×16	$\begin{bmatrix} 3\times3,\ 32\times k \\ 3\times3,\ 32\times k \end{bmatrix} \times N$
conv4	8×8	$\begin{bmatrix} 3\times3,\ 64\times k \\ 3\times3,\ 64\times k \end{bmatrix} \times N$
avg-pool	1×1	$[8 \times 8]$

Table 1: Structure of wide residual networks. Network width is determined by factor k. Original architecture [13] is equivalent to $k = 1$. Groups of convolutions are shown in brackets where N is a number of blocks in group, downsampling performed by the first layers in groups `conv3` and `conv4`. Final classification layer is omitted for clearance. In the particular example shown, the network uses a ResNet block of type $B(3,3)$.

Figure 19: Source - Wide Residual Networks, Zagoruyko and Komodakis, 2017

Classic Networks - Wide ResNet

- B(3,3) - original "basic" block
- B(3,1,3) - with one extra 1×1 layer
- B(1,3,1) - with the same dimensionality of all convolutions, *straightened* bottleneck
- B(1,3) - the network has alternating 1×1 - 3×3 convolutions everywhere
- B(3,1) - similar idea to the previous block
- B(3,1,1) - Network-in-Network style block

block type	depth	# params	time,s	CIFAR-10
$B(1,3,1)$	40	1.4M	85.8	6.06
$B(3,1)$	40	1.2M	67.5	5.78
$B(1,3)$	40	1.3M	72.2	6.42
$B(3,1,1)$	40	1.3M	82.2	5.86
$B(3,3)$	28	1.5M	67.5	5.73
$B(3,1,3)$	22	1.1M	59.9	5.78

Table 2: Test error (%, median over 5 runs) on CIFAR-10 of residual networks with $k = 2$ and different block types. Time column measures one training epoch.

l	CIFAR-10
1	6.69
2	5.43
3	5.65
4	5.93

Table 3: Test error (%, median over 5 runs) on CIFAR-10 of WRN-40-2 (2.2M) with various l.

Figure 20: Source - Wide Residual Networks, Zagoruyko and Komodakis, 2017

Classic Networks - Wide ResNet

depth	k	# params	CIFAR-10	CIFAR-100
40	1	0.6M	6.85	30.89
40	2	2.2M	5.33	26.04
40	4	8.9M	4.97	22.89
40	8	35.7M	4.66	-
28	10	36.5M	**4.17**	20.50
28	12	52.5M	4.33	**20.43**
22	8	17.2M	4.38	21.22
22	10	26.8M	4.44	20.75
16	8	11.0M	4.81	22.07
16	10	17.1M	4.56	21.59

Table 4: Test error (%) of various wide networks on CIFAR-10 and CIFAR-100 (ZCA pre-processing).

Figure 21: Source - Wide Residual Networks, Zagoruyko and Komodakis, 2017

Classic Networks - Wide ResNet

	depth-k	# params	CIFAR-10	CIFAR-100
NIN [20]			8.81	35.67
DSN [19]			8.22	34.57
FitNet [24]			8.39	35.04
Highway [28]			7.72	32.39
ELU [5]			6.55	24.28
original-ResNet[11]	110	1.7M	6.43	25.16
	1202	10.2M	7.93	27.82
stoc-depth[14]	110	1.7M	5.23	24.58
	1202	10.2M	4.91	-
pre-act-ResNet[13]	110	1.7M	6.37	-
	164	1.7M	5.46	24.33
	1001	10.2M	4.92(4.64)	22.71
WRN (ours)	40-4	8.9M	4.53	21.18
	16-8	11.0M	4.27	20.43
	28-10	36.5M	**4.00**	**19.25**

Table 5: Test error of different methods on CIFAR-10 and CIFAR-100 with moderate data augmentation (flip/translation) and mean/std normalzation. We don't use dropout for these results. In the second column k is a widening factor. Results for [13] are shown with mini-batch size 128 (as ours), and 64 in parenthesis. Our results were obtained by computing median over 5 runs.

Figure 22: Source - Wide Residual Networks, Zagoruyko and Komodakis, 2017

Classic Networks - Wide ResNet

Takeaway:

- widening consistently improves performance across residual networks of different depth
- increasing both depth and width helps until the number of parameters becomes too high and stronger regularization is needed
- **main power of residual networks is in residual blocks**, and not in extreme depth

Classic Networks - ResNeXt

Highlights

- Focuses on three central aspects inspired by previous conv nets (VGG, ResNet and Inception): **uniform blocks**, **residual connections** and **multi-branch convolutional architectures**
- Uses **grouped convolutions** to implement multi-branch *aggregated transformations*
- Considers *cardinality* (number of branches) as the important control dimension in network performance
- Strives to develop an *easy-to-use* template for building both wide and deep conv nets

Classic Networks - ResNeXt

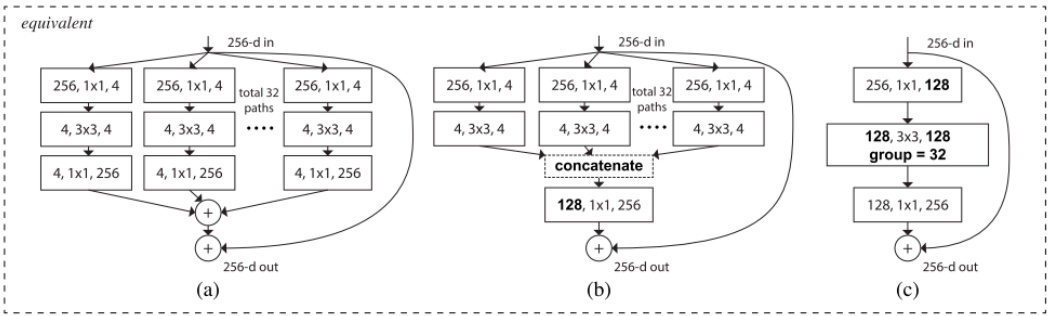


Figure 23: Equivalent building blocks of ResNeXt. (a): Aggregated residual transformations, the same as Fig. 1 right. (b): A block equivalent to (a), implemented as early concatenation. (c): A block equivalent to (a,b), implemented as grouped convolutions [24]. Notations in bold text highlight the reformulation changes. A layer is denoted as (# input channels, filter size, # output channels). Source: Aggregated Residual Transformations for Deep Neural Networks. Xie et al., 2017

Classic Networks - ResNeXt

	setting	top-1 error (%)
ResNet-50	1 × 64d	23.9
ResNeXt-50	2 × 40d	23.0
ResNeXt-50	4 × 24d	22.6
ResNeXt-50	8 × 14d	22.3
ResNeXt-50	32 × 4d	**22.2**
ResNet-101	1 × 64d	22.0
ResNeXt-101	2 × 40d	21.7
ResNeXt-101	4 × 24d	21.4
ResNeXt-101	8 × 14d	21.3
ResNeXt-101	32 × 4d	**21.2**

Table 3. Ablation experiments on ImageNet-1K. (**Top**): ResNet-50 with preserved complexity (∼4.1 billion FLOPs); (**Bottom**): ResNet-101 with preserved complexity (∼7.8 billion FLOPs). The error rate is evaluated on the single crop of 224×224 pixels.

Figure 24: Increased cardinality leads to better accuracy. 4d, 14d, etc - are widths of the bottleneck in a ResNet block

	setting	top-1 err (%)	top-5 err (%)
1× complexity references:			
ResNet-101	1 × 64d	22.0	6.0
ResNeXt-101	32 × 4d	21.2	5.6
2× complexity models follow:			
ResNet-**200** [15]	1 × 64d	21.7	5.8
ResNet-101, wider	1 × **100**d	21.3	5.7
ResNeXt-101	**2** × 64d	20.7	5.5
ResNeXt-101	**64** × 4d	**20.4**	**5.3**

Table 4. Comparisons on ImageNet-1K when the number of FLOPs is increased to 2× of ResNet-101's. The error rate is evaluated on the single crop of 224×224 pixels. The highlighted factors are the factors that increase complexity.

Figure 25: Increased cardinality is better than increased *depth* or increased *width* of the network.

Source: Aggregated Residual Transformations for Deep Neural Networks. Xie et al., 2017

Classic Networks - DenseNet

Highlights

- Simplify the **connectivity pattern** between layers as introduced in other networks (e.g. ResNet)
- Focus on obtaining **representational power** through **feature reuse**, instead of very deep or very wide network architectures
- It introduces direct connections between any two layers with the same feature-map size
- Information (feature maps) from previous volumes is **concatenated** to subsequent volumes. Concatenation amount controlled by a **growth factor**.

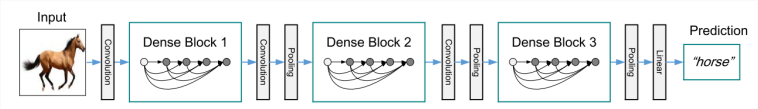


Figure 26: Source: Densely Connected Convolutional Networks, Huang et al., 2018

Classic Networks - DenseNet

Layers	Output Size	DenseNet-121	DenseNet-169	DenseNet-201	DenseNet-264
Convolution	112×112	7×7 conv, stride 2			
Pooling	56×56	3×3 max pool, stride 2			
Dense Block (1)	56×56	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 6$
Transition Layer (1)	56×56	1×1 conv			
	28×28	2×2 average pool, stride 2			
Dense Block (2)	28×28	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 12$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 12$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 12$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 12$
Transition Layer (2)	28×28	1×1 conv			
	14×14	2×2 average pool, stride 2			
Dense Block (3)	14×14	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 24$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 32$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 48$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 64$
Transition Layer (3)	14×14	1×1 conv			
	7×7	2×2 average pool, stride 2			
Dense Block (4)	7×7	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 16$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 32$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 32$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 48$
Classification Layer	1×1	7×7 global average pool			
		1000D fully-connected, softmax			

Table 1: DenseNet architectures for ImageNet. The growth rate for all the networks is $k = 32$. Note that each "conv" layer shown in the table corresponds the sequence BN-ReLU-Conv.

Figure 27: Source: Densely Connected Convolutional Networks, Huang et al., 2018

Classic Networks - DenseNet

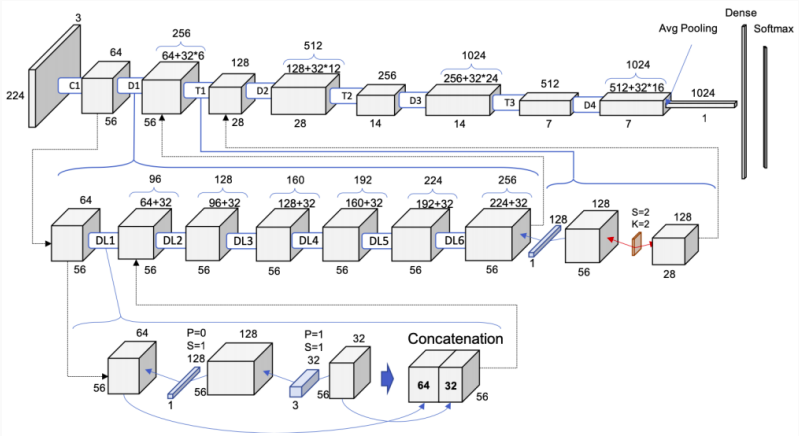


Figure 28: Full schematic representation of DenseNet-121. Source: Pablo Ruiz Blog, https://towardsdatascience.com/understanding-and-visualizing-densenets-7f688092391a

Classic Networks - DenseNet

Method	Depth	Params	C10	C10+	C100	C100+	SVHN
Network in Network [22]	-	-	10.41	8.81	35.68	-	2.35
All-CNN [32]	-	-	9.08	7.25	-	33.71	-
Deeply Supervised Net [20]	-	-	9.69	7.97	-	34.57	1.92
Highway Network [34]	-	-	-	7.72	-	32.39	-
FractalNet [17]	21	38.6M	10.18	5.22	35.34	23.30	2.01
with Dropout/Drop-path	21	38.6M	7.33	4.60	28.20	23.73	1.87
ResNet [11]	110	1.7M	-	6.61	-	-	-
ResNet (reported by [13])	110	1.7M	13.63	6.41	44.74	27.22	2.01
ResNet with Stochastic Depth [13]	110	1.7M	11.66	5.23	37.80	24.58	1.75
	1202	10.2M	-	4.91	-	-	-
Wide ResNet [42]	16	11.0M	-	4.81	-	22.07	-
	28	36.5M	-	4.17	-	20.50	-
with Dropout	16	2.7M	-	-	-	-	1.64
ResNet (pre-activation) [12]	164	1.7M	11.26*	5.46	35.58*	24.33	-
	1001	10.2M	10.56*	4.62	33.47*	22.71	-
DenseNet ($k = 12$)	40	1.0M	**7.00**	5.24	**27.55**	24.42	1.79
DenseNet ($k = 12$)	100	7.0M	**5.77**	**4.10**	**23.79**	**20.20**	1.67
DenseNet ($k = 24$)	100	27.2M	**5.83**	**3.74**	**23.42**	**19.25**	**1.59**
DenseNet-BC ($k = 12$)	100	0.8M	**5.92**	4.51	**24.15**	22.27	1.76
DenseNet-BC ($k = 24$)	250	15.3M	**5.19**	**3.62**	**19.64**	**17.60**	1.74
DenseNet-BC ($k = 40$)	190	25.6M	-	**3.46**	-	**17.18**	-

Table 2: Error rates (%) on CIFAR and SVHN datasets. k denotes network's growth rate. Results that surpass all competing methods are **bold** and the overall best results are **blue**. "+" indicates standard data augmentation (translation and/or mirroring). $*$ indicates results run by ourselves. All the results of DenseNets without data augmentation (C10, C100, SVHN) are obtained using Dropout. DenseNets achieve lower error rates while using fewer parameters than ResNet. Without data augmentation, DenseNet performs better by a large margin.

Figure 29: Source: Densely Connected Convolutional Networks, Huang et al., 2018

Summary CNN Architectures

- Many of the mentioned architectures are available in "model zoos" (e.g. PyTorch Model Zoo)
- ResNets are still a good default to use
- Research into CNNs has sought to analyze *representational power gains* in terms of:
  - Depth of the network (deeper is *usually* better)
  - Width of convolutional layers/blocks (how many channels are learned) – trading depth for width *can* lead to improved performance, provided proper data (e.g. WideResNet)
  - Forced learning of multiple *"perspectives"* of the same data (e.g. GoogleNet, ResNeXt) *can* lead to improved performance
  - Feature reuse (DenseNet) *can* increase representational power

Regularization in Conv Nets

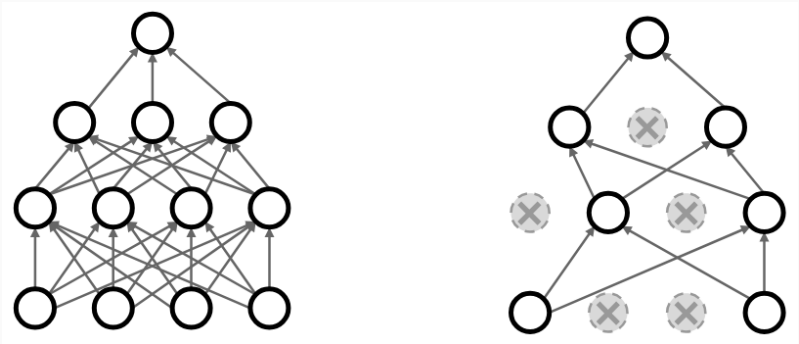
Regularization in Deep Networks

Methods that *effect* regularization in deep neural networks

1. Regularization of the weights (e.g L2, weight decay – see class on Optimization for NN)
2. Dropout
3. Data Augmentation
4. Stochastic Depth
5. Cutout / MixUp

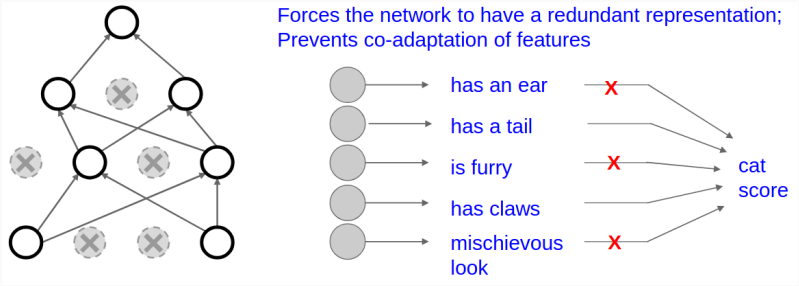
Regularization: Dropout

- In each forward pass, randomly set some neurons (activations) to 0
- Probability of *dropping* is a hyperparameter (common values are between [0.1, 0.5])



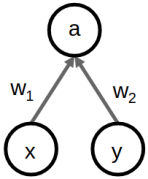
Regularization: Dropout

Dropout intuition: why should it work? What does it try to do?



Regularization: Dropout

What happens at test time? (neurons are **only** dropped at train time)



Consider a single neuron.

At test time we have: $E[a] = w_1 x + w_2 y$

During training we have:

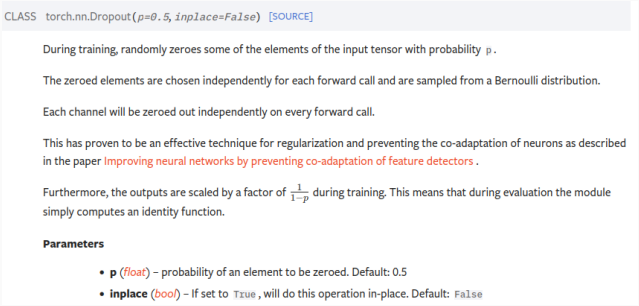
$$\begin{aligned} E[a] =& \frac{1}{4}(w_1 x + w_2 y) + \frac{1}{4}(w_1 x + 0y) \\ &+ \frac{1}{4}(0x + 0y) + \frac{1}{4}(0x + w_2 y) \\ =& \frac{1}{2}(w_1 x + w_2 y) \end{aligned}$$

- We must **scale** the activations such that for each neuron **output at test time** = **expected output at train time**

Regularization: Dropout

We must **scale** the activations such that for each neuron **output at test time = expected output at train time**

- More common in practice - inverted dropout: scale down the activations of neurons **that remain active** by $\frac{1}{1-p}$ (where p = probability of dropout)



CLASS torch.nn.Dropout(*p=0.5, inplace=False*) [SOURCE]

During training, randomly zeroes some of the elements of the input tensor with probability `p`.

The zeroed elements are chosen independently for each forward call and are sampled from a Bernoulli distribution.

Each channel will be zeroed out independently on every forward call.

This has proven to be an effective technique for regularization and preventing the co-adaptation of neurons as described in the paper Improving neural networks by preventing co-adaptation of feature detectors .

Furthermore, the outputs are scaled by a factor of $\frac{1}{1-p}$ during training. This means that during evaluation the module simply computes an identity function.

Parameters

- **p** (*float*) – probability of an element to be zeroed. Default: 0.5
- **inplace** (*bool*) – If set to `True`, will do this operation in-place. Default: `False`

Regularization: Data Augmentation

Data augmentation: **transform** the input synthetically (including by adding noise), such that it *may* represent *possible* real input not captured in the existing dataset

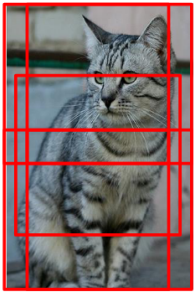
Useful (creative) augmentations are **problem dependent**!
Typical examples for images:

- Translation, Rotation, Stretching, Shearing
- Random crops and scales
- Vertical or Horizontal Flips
- Lens distortions (e.g. gaussian, motion blur, fish-eye),
- Color Jittering, Hue/Saturation changes, Grey Scale Conversion

Some are so common they are bundled up in frameworks (e.g. Albumentations).

Data Augmentation

Random crops and scales



Training: sample random crops / scales

ResNet:

1. Pick random L in range [256, 480]
2. Resize training image, short side = L
3. Sample random 224 x 224 patch

Testing: average a fixed set of crops

ResNet:

1. Resize image at 5 scales: {224, 256, 384, 480, 640}
2. For each size, use 10 224 x 224 crops: 4 corners + center, + flips

Data Augmentation

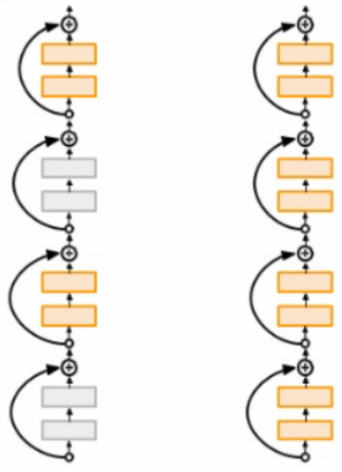
Color Jitter

Simple: Randomize
contrast and brightness



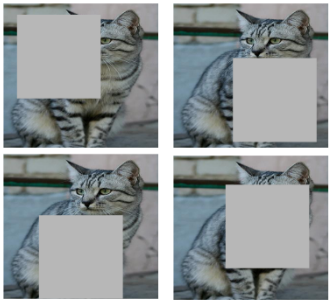
Regularization: Stochastic Depth

- At training time: skip some layers in the network
- At test time: Use all the layers



Regularization: Cutout / Mixup

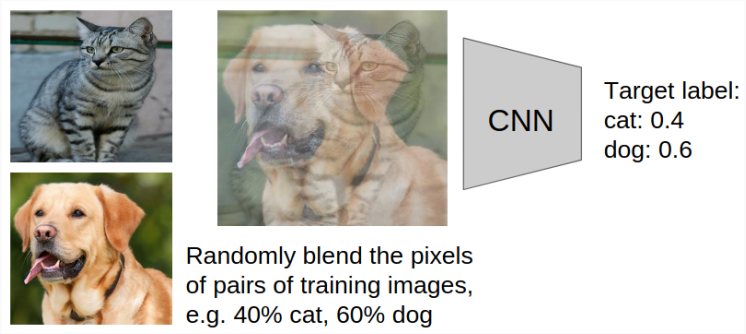
- At training time: set random image **regions** to zero
- At test time: Use full image
- Works well for small datasets (e.g. the size of CIFAR - 60K images), less needed for large datasets (e.g. the size of ImageNet - 1.3M images)



Regularization: Cutout / Mixup

Mixup (Zhang et al., "Mixup: Beyond Empirical Risk Minimization", ICLR 2018):

- At train time: train on random **blends** of images
- At test time: use original images



Transfer Learning

Transfer Learning

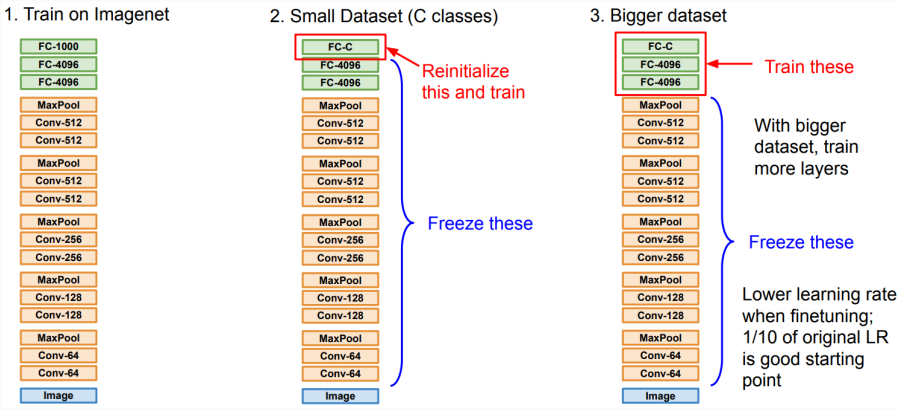
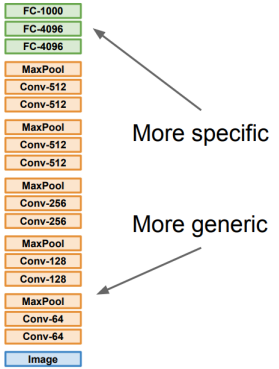


Figure 30: Source: Stanford CS231n Lecture 9, 2019

Transfer Learning



	very similar dataset	very different dataset
very little data	Use Linear Classifier on top layer	You're in trouble… Try linear classifier from different stages
quite a lot of data	Finetune a few layers	Finetune a larger number of layers

Figure 31: Source: Stanford CS231n Lecture 9, 2019

Transfer Learning

- Transfer Learning with CNNs is *the norm, not the exception*
  - CNNs trained on ImageNet used in object detection (e.g. Fast R-CNN), image captioning (CNN + RNN)

- Takeaway: if you have some dataset of interest, but it has significantly less than 1 million images
  - Find a large dataset with similar data, train big ConvNet there
  - Transfer learn to your dataset
  - Deep Learning Frameworks (e.g. Torch Vision) already provide a "Model Zoo" of pre-trained models (usually on ImageNet), so you don't need to train your own from scratch